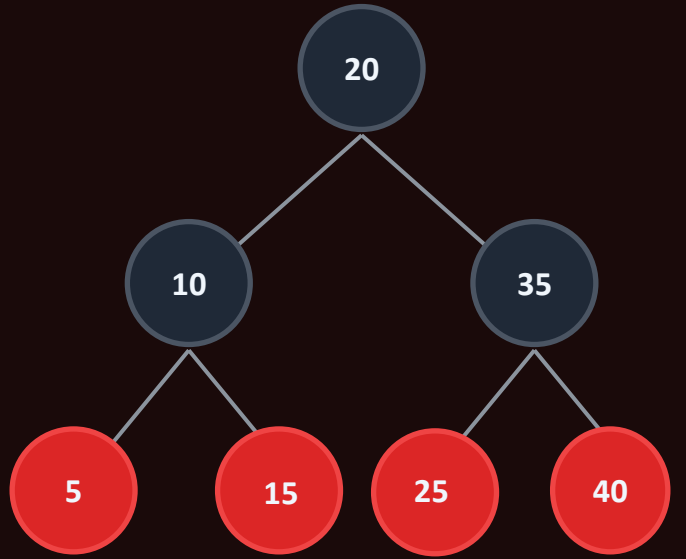
DATA STRUCTURES & ALGORITHMS

Red-Black Tree



Overview

A Red-Black Tree is a self-balancing Binary Search Tree. Each node has an extra attribute "colour" (Red or Black). Supports insertion & deletion in O(log n) time.

AVL VS RED-BLACK

AVL TREE	RED-BLACK TREE
Strict BST	Loose BST
More complex insertion/deletion	Less complex insertion/deletion
Node: Left | Key | Right	Node: Left | Colour | Key | Right

Properties

1. Each node has a colour: either Red or Black.
2. The root of the entire tree is always Black.
3. A red node cannot have a red child (no two consecutive red nodes on any path).
4. NULL pointers are treated as Black .
5. ★★ Black-Height Rule: Total black nodes on any left branch = total black nodes on any right branch.

Insertion in Red-Black Tree

Time Complexity --- O(log n)

Always insert a new node with colour RED.

Insert 10

RULE 2 – ROOT PROPERTY VIOLATION

If the root becomes Red after an operation, simply recolour it to Black.

Insertion in Red-Black Tree

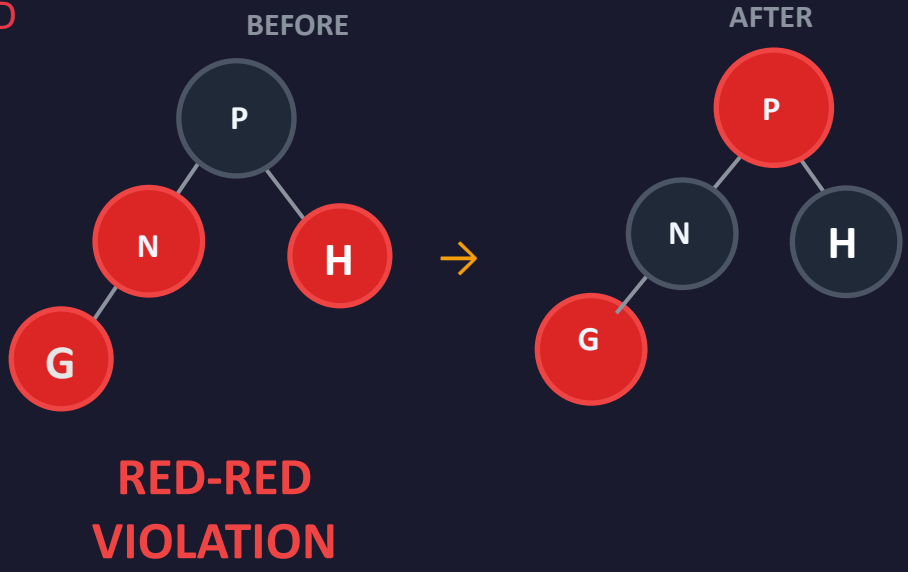
CASE A – RED-RED VIOLATION

If the new node's parent is also red, a violation occurs. Check the uncle:

UNCLE IS RED

Recolour parent & uncle → Black

grandparent → Red



Insertion in Red-Black Tree

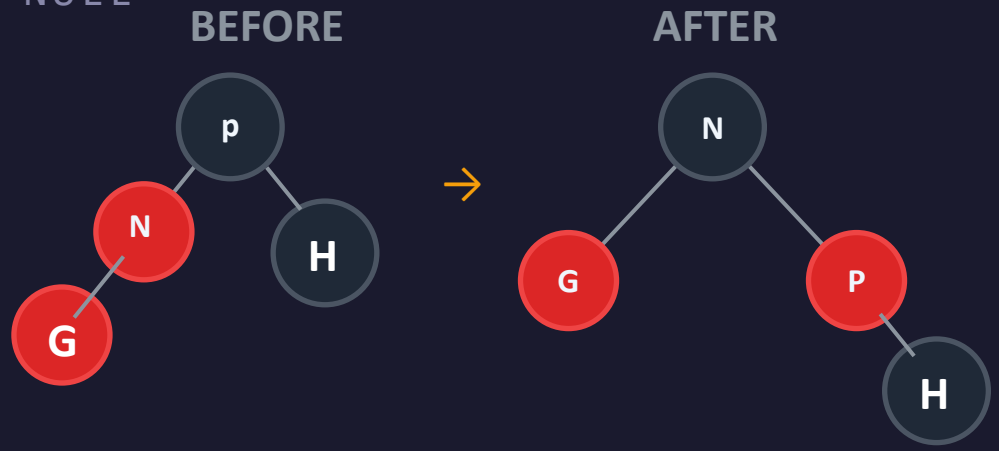
CASE A – RED-RED VIOLATION

If the new node's parent is also red, a violation occurs. Check the uncle:

UNCLE IS BLACK / NULL

Perform rotation (LL/LR/RR/RL)

then recolour



LL Case (Right Rotate)

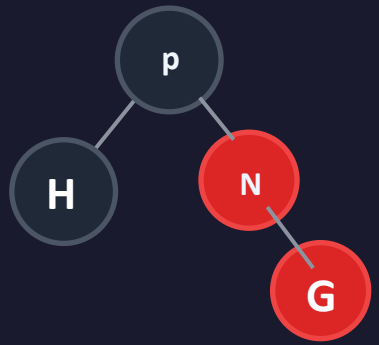
Recolour(after rotation)

UNCLE IS BLACK / NULL

Perform rotation (LL/LR/RR/RL)

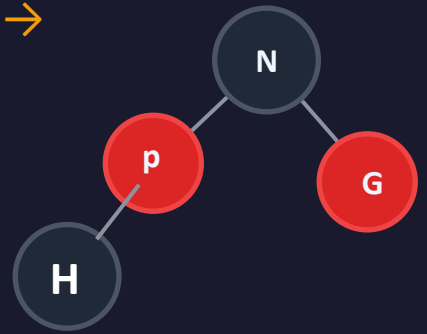
then recolour.

BEFORE



RR Case (Left Rotate)

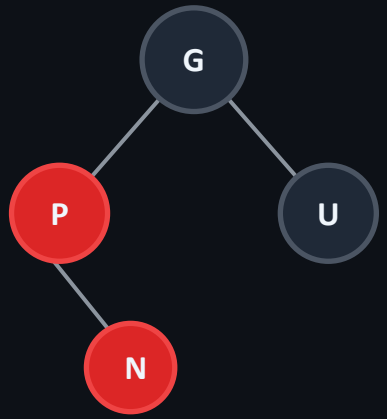
AFTER



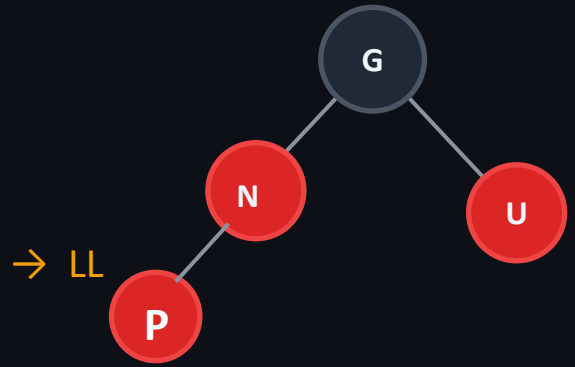
Recolour(after rotation)

LR Case (Left rotate parent (→LL), then LL fix)

Step 1: Left rotate p

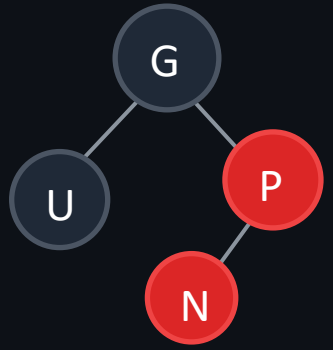


Step 2: Apply LL fix

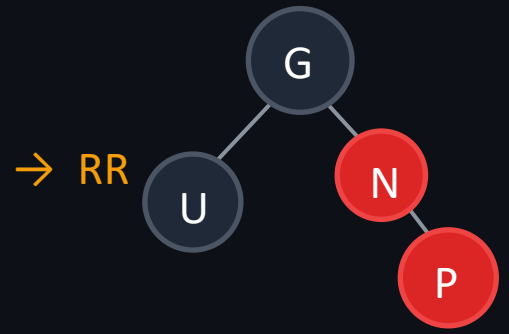


RL Case (Right rotate parent , then RR fix)

Step 1: right rotate p

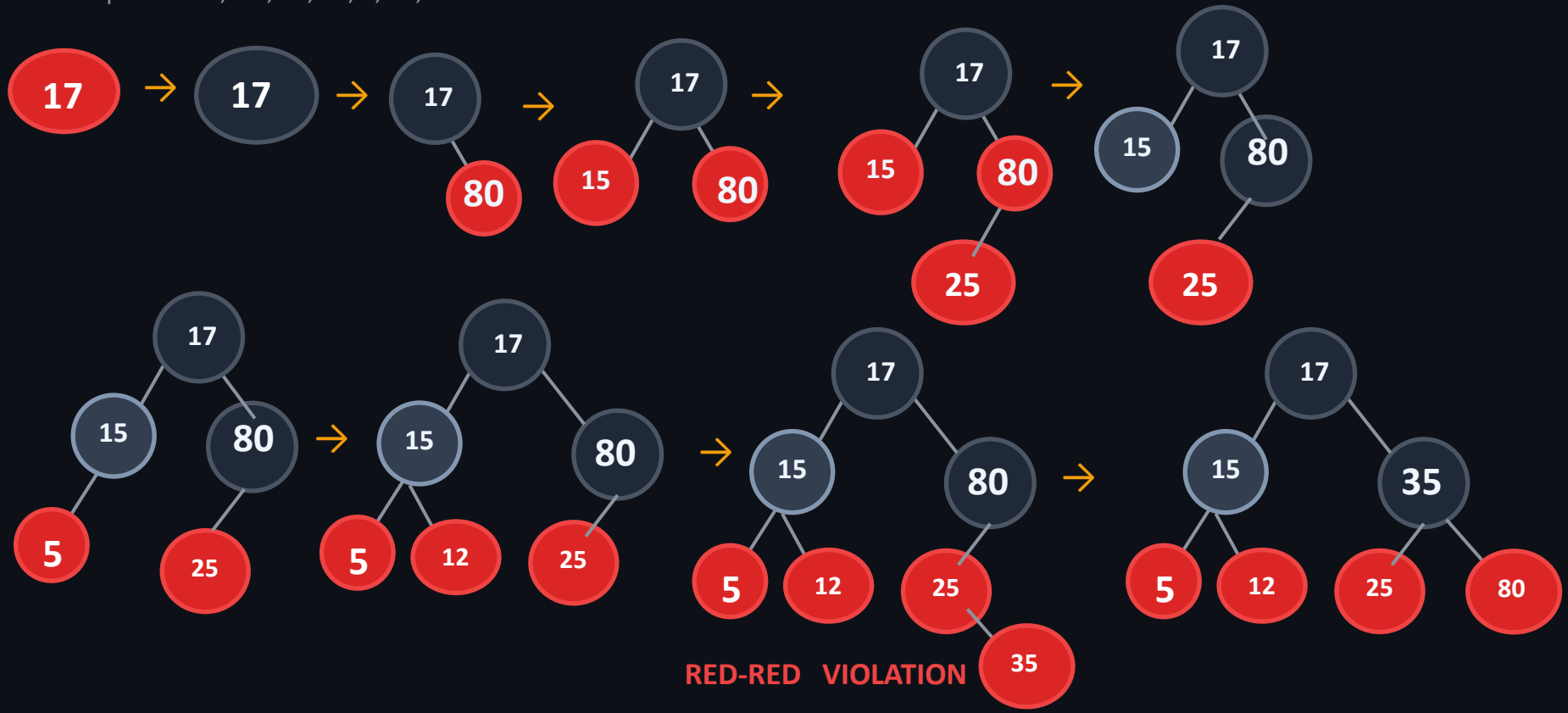


Step 2: Apply LL fix



Insertion Example —

Sequence: 17, 80, 15, 25, 5, 12, 35

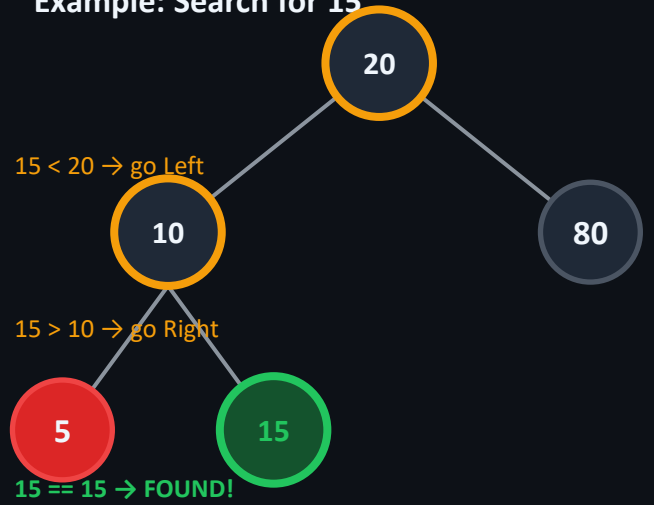


Search in Red-Black Tree

Search ignores colours entirely — it's identical to a standard BST search.

1. **Start at Root**
   Begin search at the root node.
2. **Compare Key**
   If target == node key → Found! Return node.
3. **Go Left or Right**
   target < node → go Left child
   target > node → go Right child
4. **NULL = Not Found**
   If you reach a NULL pointer, the key does not exist in the tree.

Example: Search for 15



Time Complexity: Best Case: O(1) — key is at root Average & Worst Case: O(log n)

Deletion in Red-Black Tree ★★★

① Deleting a RED Node

Leaf Node

Simply remove.

Internal Node

Swap with inorder predecessor/successor (no colour change), delete leaf copy.

② Deleting a BLACK Node

Leaf Node

Replace with NULL, mark as Double Black (DB). Must resolve DB.

Internal Node

Same as red: swap with inorder pred/succ, then handle leaf deletion.

Time Complexity --- O(log n)

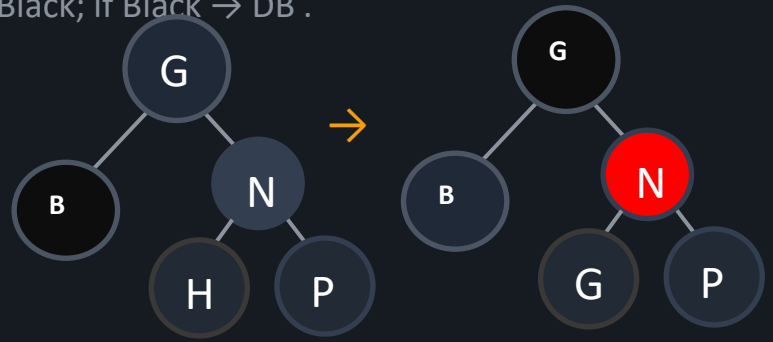
Resolving Double Black (DB)

DB is at ROOT

Simply recolour root from DB → Black.

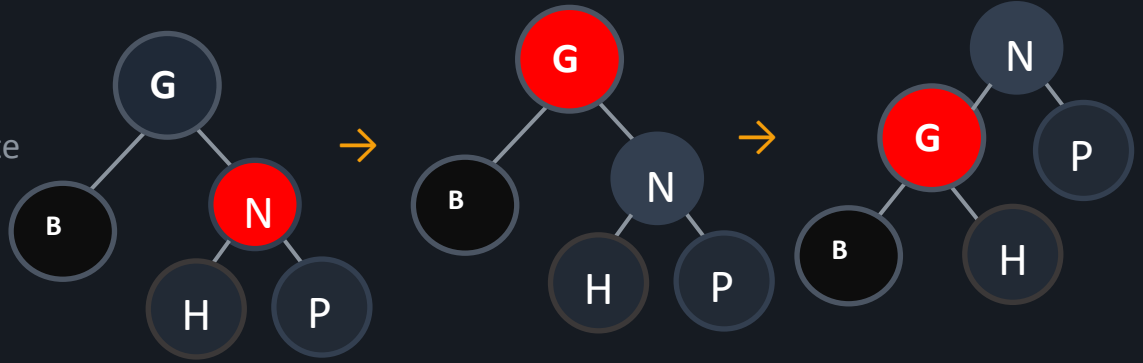
Sibling BLACK, both nephews BLACK

Sibling → Red. Parent→Black: if Red → Black; if Black → DB .



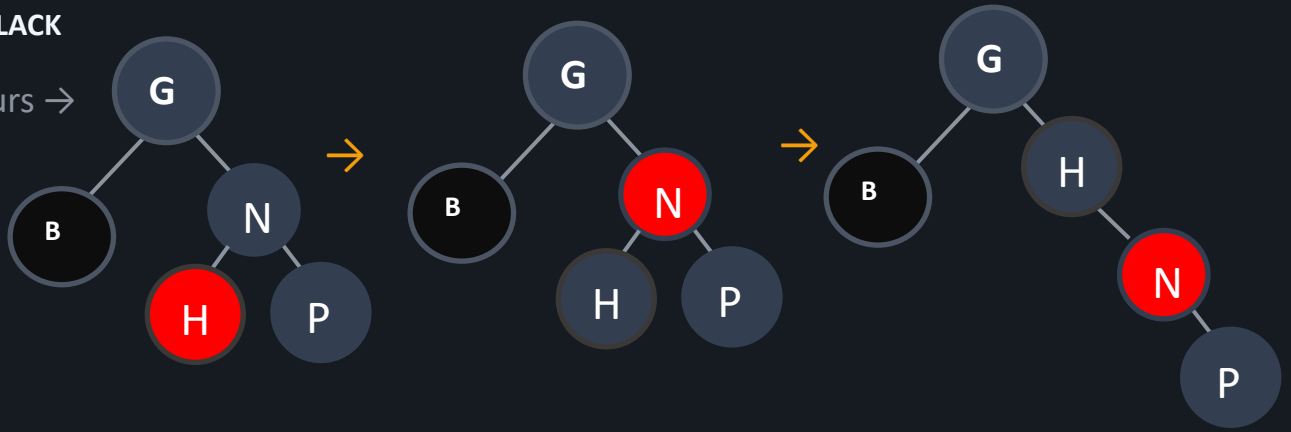
Sibling is RED

swap parent & sibling colours → Rotate parent toward DB



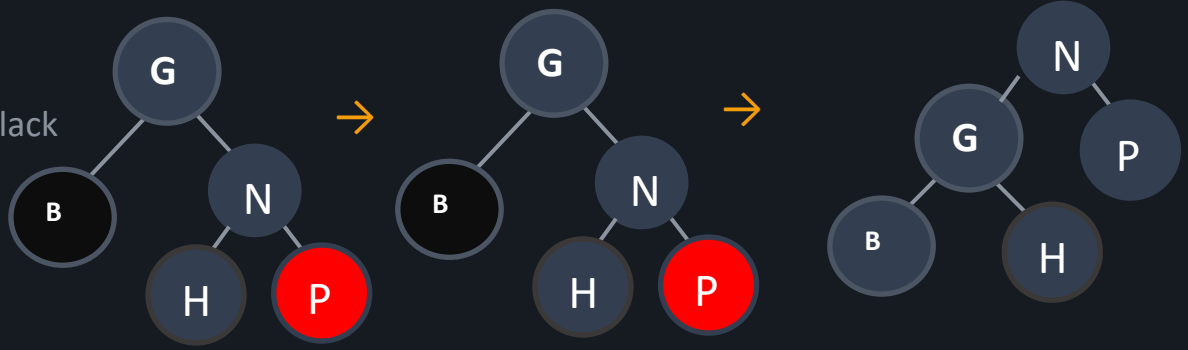
Sibling BLACK, near nephew RED, far BLACK

Swap sibling & near-nephew colours → rotate sibling toward Black child .



Sibling BLACK, far nephew RED

Swap parent & sibling colours → rotate parent toward DB → then make DB → Black →then far nephew → Black.



Conclusion

SPEED

Performance

O(log n) insert, delete, and search — always.
Never degrades like an unbalanced BST.

USAGE

Real-World Usage

C++ STL map & set, Java TreeMap & TreeSet,
used for indexing and fast searching

CONCEPT

Core CS Concept

Understanding RBT builds intuition for self-balancing structures used everywhere in systems.

KEY TAKEAWAY: *Red-Black Trees provide an efficient balance between speed and structure, making them ideal for dynamic data applications..*